Wallet Management App – Approach & Specification

Internship Name: Farida Bey

Project: Wallet Management System (Android + Spring Boot + PostgreSQL)

1) Product Scope (MVP)

Goal: Build a wallet management application that allows users to add mobile wallets and transfer money using a recipient mobile number.

Core capabilities:

- Register/Login users
- Create (add) mobile wallet(s) identified by a mobile number
- View wallet balance and transaction history
- Send money by entering recipient mobile number + amount + optional note
- Maintain secure, consistent transfers (atomic debit/credit)

2) Core Entities & Rules

Core entities:

- User
- Mobile Wallet
- Transaction

Key rules:

- A mobile wallet is uniquely identified by its mobile number.
- Transfers must be atomic (debit + credit in a single database transaction).
- Sender must have sufficient balance; recipient wallet must exist and be ACTIVE.

3) Android App Features

3.1 Screens

- Login / Register (phone + password)
- Home (current balance + quick actions)
- Add Mobile Wallet (enter mobile number + optional wallet name)
- Send Money (recipient mobile number, amount, optional note, confirm)
- Transaction History (All / Sent / Received filters)

3.2 UX Notes

- Show clear errors: Wallet not found, Invalid mobile number, Insufficient balance.
- Use loading states and success receipts after transfer.
- Mask sensitive data in UI where relevant; avoid logging tokens.

4) Backend Design (Spring Boot + PostgreSQL)

Recommended architecture: controller → service → repository (Spring Data JPA).

Use JWT authentication and role-based access if needed (optional for MVP).

4.1 Transfer Service Logic (Important)

1. Identify sender wallet (default wallet or selected).
2. Find recipient wallet by mobile number.
3. Validate amount, sender balance, wallet statuses, and prevent self-transfer.
4. Execute with @Transactional to ensure atomic debit/credit.
5. Update balances and write transaction records.
6. Return receipt data (transaction id, timestamp, updated balances).

5) Validation – Egyptian Mobile Number

Egyptian mobile numbers are typically 11 digits and commonly start with 01 followed by one of: 0, 1, 2, or 5, then 8 digits.

Recommended regex (server + client):

^01[0125][0-9]{8}$

Examples:

- Valid: 01012345678
- Valid: 01123456789
- Valid: 01234567890

- Valid: 01512345678
- Invalid: 01312345678 (unsupported prefix)
- Invalid: 0101234567 (too short)
- Invalid: 010123456789 (too long)

Implementation notes:

- Apply the same regex on Android (client-side) and Spring Boot (server-side).
- Server-side validation is mandatory even if client-side validation exists.
- Store mobile numbers consistently (no spaces; standardize leading zeros).

6) Final Deliverables (Demo Ready)

- Android APK + source repo
- Spring Boot API + source repo
- PostgreSQL migrations (Flyway/Liquibase)
- Swagger/OpenAPI docs + Postman collection
- Final demo script + README